

Hands on DAY31 – Keerthivasan R

Problem Statement: Student Management System (CRUD)

Background

A small coaching institute wants to maintain student records digitally instead of using manual registers. The system should allow administrators to manage student information efficiently.

Objective

Develop a simple database-driven application that performs **CRUD operations (Create, Read, Update, Delete)** on student records.

Functional Requirements

The system should allow the user to:

1. Create

- a. Add a new student record.
- b. Store details such as:
 - i. Student ID (Primary Key)
 - ii. Name
 - iii. Email
 - iv. Phone Number
 - v. Course
 - vi. Date of Admission

2. Read

- a. View all student records in a list format.
- b. Search student by ID or Name.
- c. View detailed information of a single student.

3. Update

- a. Edit existing student details.
- b. Save updated information in the database.

4. Delete

- a. Remove a student record from the system.
- b. Ask for confirmation before deletion.

Database Requirements

Table: Students

Column Name	Data Type	Constraints
StudentId	int	Primary Key, Identity
Name	varchar(100)	Not Null
Email	varchar(100)	Unique
Phone	varchar(15)	Not Null
Course	varchar(50)	Not Null
AdmissionDate	datetime	Not Null

Technical Constraints

- Use SQL Server as the database.
- Use Entity Framework (Database First or Code First).
- Implement MVC architecture.
- Apply basic validation (Required fields, Email format).
- Use proper routing for CRUD actions.

Expected Outcome

- User-friendly interface.
- Properly connected database.
- All four CRUD operations working successfully.
- Data validation and error handling implemented.

CODE:

Model/ Student.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace StudentManagementSystem.Models
{
    public class Student
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int StudentId { get; set; }

        [Required(ErrorMessage = "Name is required")]
        [StringLength(100, MinimumLength = 2, ErrorMessage = "Name must be between 2
and 100 characters")]
        [Display(Name = "Full Name")]
        public string Name { get; set; } = string.Empty;

        [Required(ErrorMessage = "Email is required")]
        [EmailAddress(ErrorMessage = "Invalid email format")]
        [StringLength(100)]
        [Display(Name = "Email Address")]
        public string Email { get; set; } = string.Empty;

        [Required(ErrorMessage = "Phone number is required")]
        [StringLength(15, MinimumLength = 10, ErrorMessage = "Phone number must be
10-15 digits")]
        [Phone(ErrorMessage = "Invalid phone number")]
        [Display(Name = "Phone Number")]
        public string Phone { get; set; } = string.Empty;

        [Required(ErrorMessage = "Course is required")]
        [StringLength(50)]
        [Display(Name = "Course Name")]
        public string Course { get; set; } = string.Empty;

        [Required(ErrorMessage = "Admission date is required")]
        [DataType(DataType.Date)]
        [Display(Name = "Admission Date")]
        [DisplayFormat(DataFormatString = "{0:yyyy-MM-dd}", ApplyFormatInEditMode =
true)]
        public DateTime AdmissionDate { get; set; } = DateTime.Now;
    }
}
```

Data / ApplicationDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using StudentManagementSystem.Models;

namespace StudentManagementSystem.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }
    }
}
```

```

    }

    public DbSet<Student> Students { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // Configure Email as Unique
        modelBuilder.Entity<Student>()
            .HasIndex(s => s.Email)
            .IsUnique();

        // Seed some initial data
        modelBuilder.Entity<Student>().HasData(
            new Student
            {
                StudentId = 1,
                Name = "John Doe",
                Email = "john@example.com",
                Phone = "9876543210",
                Course = "Computer Science",
                AdmissionDate = new DateTime(2024, 1, 15)
            },
            new Student
            {
                StudentId = 2,
                Name = "Jane Smith",
                Email = "jane@example.com",
                Phone = "9876543211",
                Course = "Mathematics",
                AdmissionDate = new DateTime(2024, 1, 20)
            }
        );
    }
}

```

Controllers/ StudentController.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using StudentManagementSystem.Data;
using StudentManagementSystem.Models;

namespace StudentManagementSystem.Controllers
{
    public class StudentController : Controller
    {
        private readonly ApplicationDbContext _context;

        public StudentController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: Student/Index - List all students
        public async Task<IActionResult> Index(string searchString)
        {
            var students = from s in _context.Students
                           select s;

```

```

        if (!string.IsNullOrEmpty(searchString))
        {
            students = students.Where(s => s.Name.Contains(searchString) ||
s.StudentId.ToString().Contains(searchString));
        }

        ViewData["CurrentFilter"] = searchString;
        return View(await students.ToListAsync());
    }

    // GET: Student/Details/5 - View student details
    public async Task<IActionResult> Details(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var student = await _context.Students
            .FirstOrDefaultAsync(m => m.StudentId == id);

        if (student == null)
        {
            return NotFound();
        }

        return View(student);
    }

    // GET: Student/Create - Show create form
    public IActionResult Create()
    {
        return View();
    }

    // POST: Student/Create - Add new student
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult>
Create([Bind("Name,Email,Phone,Course,AdmissionDate")] Student student)
    {
        if (ModelState.IsValid)
        {
            // Check if email already exists
            var existingStudent = await _context.Students
                .FirstOrDefaultAsync(s => s.Email == student.Email);

            if (existingStudent != null)
            {
                ModelState.AddModelError("Email", "This email is already
registered");
                return View(student);
            }

            _context.Add(student);
            await _context.SaveChangesAsync();
            TempData["SuccessMessage"] = "Student added successfully!";
            return RedirectToAction(nameof(Index));
        }
    }

```

```

        return View(student);
    }

    // GET: Student/Edit/5 - Show edit form
    public async Task<IActionResult> Edit(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var student = await _context.Students.FindAsync(id);
        if (student == null)
        {
            return NotFound();
        }
        return View(student);
    }

    // POST: Student/Edit/5 - Update student
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(int id,
[Bind("StudentId,Name,Email,Phone,Course,AdmissionDate")] Student student)
    {
        if (id != student.StudentId)
        {
            return NotFound();
        }

        if (ModelState.IsValid)
        {
            try
            {
                // Check if email exists for another student
                var existingStudent = await _context.Students
                    .FirstOrDefaultAsync(s => s.Email == student.Email &&
s.StudentId != student.StudentId);

                if (existingStudent != null)
                {
                    ModelState.AddModelError("Email", "This email is already
registered to another student");
                    return View(student);
                }

                _context.Update(student);
                await _context.SaveChangesAsync();
                TempData["SuccessMessage"] = "Student updated successfully!";
            }
            catch (DbUpdateConcurrencyException)
            {
                if (!StudentExists(student.StudentId))
                {
                    return NotFound();
                }
                else
                {
                    throw;
                }
            }
        }
    }

```

```

        return RedirectToAction(nameof(Index));
    }
    return View(student);
}

// GET: Student/Delete/5 - Show delete confirmation
public async Task<IActionResult> Delete(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var student = await _context.Students
        .FirstOrDefaultAsync(m => m.StudentId == id);
    if (student == null)
    {
        return NotFound();
    }

    return View(student);
}

// POST: Student/Delete/5 - Delete student
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var student = await _context.Students.FindAsync(id);
    if (student != null)
    {
        _context.Students.Remove(student);
        await _context.SaveChangesAsync();
        TempData["SuccessMessage"] = "Student deleted successfully!";
    }

    return RedirectToAction(nameof(Index));
}

private bool StudentExists(int id)
{
    return _context.Students.Any(e => e.StudentId == id);
}
}
}

```

Views / Students / Index.cshtml

```

@model IEnumerable<StudentManagementSystem.Models.Student>

@{
    ViewData["Title"] = "Student List";
}

<h2>Student Management System</h2>

@if (TempData["SuccessMessage"] != null)
{
    <div class="alert alert-success alert-dismissible fade show" role="alert">
        @TempData["SuccessMessage"]
        <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    </div>
}

```

```

    </div>
}

<p>
    <a asp-action="Create" class="btn btn-primary">Add New Student</a>
</p>

<form asp-action="Index" method="get" class="mb-3">
    <div class="row">
        <div class="col-md-6">
            <div class="input-group">
                <input type="text" name="searchString"
value="@ViewData["CurrentFilter"]"
                class="form-control" placeholder="Search by Name or ID" />
                <button type="submit" class="btn btn-secondary">Search</button>
                <a asp-action="Index" class="btn btn-outline-secondary">Clear</a>
            </div>
        </div>
    </div>
</form>

<table class="table table-striped table-hover">
    <thead class="table-dark">
        <tr>
            <th>@Html.DisplayNameFor(model => model.StudentId)</th>
            <th>@Html.DisplayNameFor(model => model.Name)</th>
            <th>@Html.DisplayNameFor(model => model.Email)</th>
            <th>@Html.DisplayNameFor(model => model.Phone)</th>
            <th>@Html.DisplayNameFor(model => model.Course)</th>
            <th>@Html.DisplayNameFor(model => model.AdmissionDate)</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var item in Model)
        {
            <tr>
                <td>@Html.DisplayFor(modelItem => item.StudentId)</td>
                <td>@Html.DisplayFor(modelItem => item.Name)</td>
                <td>@Html.DisplayFor(modelItem => item.Email)</td>
                <td>@Html.DisplayFor(modelItem => item.Phone)</td>
                <td>@Html.DisplayFor(modelItem => item.Course)</td>
                <td>@item.AdmissionDate.ToString("dd-MMM-yyyy")</td>
                <td>
                    <a asp-action="Details" asp-route-id="@item.StudentId"
class="btn btn-info btn-sm">Details</a>
                    <a asp-action="Edit" asp-route-id="@item.StudentId" class="btn
btn-warning btn-sm">Edit</a>
                    <a asp-action="Delete" asp-route-id="@item.StudentId" class="btn
btn-danger btn-sm">Delete</a>
                </td>
            </tr>
        }
    </tbody>
</table>

@if (!Model.Any())
{
    <div class="alert alert-info">
        No students found. Click "Add New Student" to add your first student.
    </div>
}

```



```
}
```

```
Views/ Students/ Create.cshtml
```

```
@model StudentManagementSystem.Models.Student
```

```
@{  
    ViewData["Title"] = "Add New Student";  
}
```

```
<h2>Add New Student</h2>
```

```
<form asp-action="Create" method="post">  
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>  
  
    <div class="row mb-3">  
        <div class="col-md-6">  
            <label asp-for="Name" class="form-label"></label>  
            <input asp-for="Name" class="form-control" />  
            <span asp-validation-for="Name" class="text-danger"></span>  
        </div>  
        <div class="col-md-6">  
            <label asp-for="Email" class="form-label"></label>  
            <input asp-for="Email" class="form-control" type="email" />  
            <span asp-validation-for="Email" class="text-danger"></span>  
        </div>  
    </div>  
  
    <div class="row mb-3">  
        <div class="col-md-6">  
            <label asp-for="Phone" class="form-label"></label>  
            <input asp-for="Phone" class="form-control" />  
            <span asp-validation-for="Phone" class="text-danger"></span>  
        </div>  
        <div class="col-md-6">  
            <label asp-for="Course" class="form-label"></label>  
            <input asp-for="Course" class="form-control" />  
            <span asp-validation-for="Course" class="text-danger"></span>  
        </div>  
    </div>  
  
    <div class="row mb-3">  
        <div class="col-md-6">  
            <label asp-for="AdmissionDate" class="form-label"></label>  
            <input asp-for="AdmissionDate" class="form-control" type="date" />  
            <span asp-validation-for="AdmissionDate" class="text-danger"></span>  
        </div>  
    </div>  
    <div class="row">  
        <div class="col-md-12">  
            <button type="submit" class="btn btn-primary">Save Student</button>  
            <a asp-action="Index" class="btn btn-secondary">Cancel</a>  
        </div>  
    </div>  
</form>
```

```
@section Scripts {  
    @{  
        await Html.RenderPartialAsync("_ValidationScriptsPartial");  
    }  
}
```

Views / Students / Edits.cshtml

```
@model StudentManagementSystem.Models.Student

@{
    ViewData["Title"] = "Edit Student";
}

<h2>Edit Student</h2>

<form asp-action="Edit" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <input type="hidden" asp-for="StudentId" />

    <div class="row mb-3">
        <div class="col-md-6">
            <label asp-for="Name" class="form-label"></label>
            <input asp-for="Name" class="form-control" />
            <span asp-validation-for="Name" class="text-danger"></span>
        </div>
        <div class="col-md-6">
            <label asp-for="Email" class="form-label"></label>
            <input asp-for="Email" class="form-control" type="email" />
            <span asp-validation-for="Email" class="text-danger"></span>
        </div>
    </div>

    <div class="row mb-3">
        <div class="col-md-6">
            <label asp-for="Phone" class="form-label"></label>
            <input asp-for="Phone" class="form-control" />
            <span asp-validation-for="Phone" class="text-danger"></span>
        </div>
        <div class="col-md-6">
            <label asp-for="Course" class="form-label"></label>
            <input asp-for="Course" class="form-control" />
            <span asp-validation-for="Course" class="text-danger"></span>
        </div>
    </div>

    <div class="row mb-3">
        <div class="col-md-6">
            <label asp-for="AdmissionDate" class="form-label"></label>
            <input asp-for="AdmissionDate" class="form-control" type="date" />
            <span asp-validation-for="AdmissionDate" class="text-danger"></span>
        </div>
    </div>

    <div class="row">
        <div class="col-md-12">
            <button type="submit" class="btn btn-primary">Update Student</button>
            <a asp-action="Index" class="btn btn-secondary">Cancel</a>
        </div>
    </div>
</form>

@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }
}
```

Views /Students/ Details.cshtml

```
@model StudentManagementSystem.Models.Student

@{
    ViewData["Title"] = "Student Details";
}

<h2>Student Details</h2>

<div class="card">
    <div class="card-body">
        <dl class="row">
            <dt class="col-sm-3">Student ID</dt>
            <dd class="col-sm-9">@Model.StudentId</dd>

            <dt class="col-sm-3">Name</dt>
            <dd class="col-sm-9">@Model.Name</dd>

            <dt class="col-sm-3">Email</dt>
            <dd class="col-sm-9">@Model.Email</dd>

            <dt class="col-sm-3">Phone</dt>
            <dd class="col-sm-9">@Model.Phone</dd>

            <dt class="col-sm-3">Course</dt>
            <dd class="col-sm-9">@Model.Course</dd>

            <dt class="col-sm-3">Admission Date</dt>
            <dd class="col-sm-9">@Model.AdmissionDate.ToString("dd MMMM yyyy")</dd>
        </dl>
    </div>
</div>

<p class="mt-3">
    <a asp-action="Edit" asp-route-id="@Model.StudentId" class="btn btn-warning">Edit</a>
    <a asp-action="Index" class="btn btn-secondary">Back to List</a>
</p>
```

Views / Student / Delete.cshtml

```
@model StudentManagementSystem.Models.Student

@{
    ViewData["Title"] = "Delete Student";
}

<h2>Delete Student</h2>

<div class="alert alert-danger">
    <h4>Are you sure you want to delete this student?</h4>
    <p>This action cannot be undone.</p>
</div>

<div class="card">
    <div class="card-body">
        <dl class="row">
            <dt class="col-sm-3">Student ID</dt>
            <dd class="col-sm-9">@Model.StudentId</dd>
        </dl>
    </div>
</div>
```

```

<dt class="col-sm-3">Name</dt>
<dd class="col-sm-9">@Model.Name</dd>

<dt class="col-sm-3">Email</dt>
<dd class="col-sm-9">@Model.Email</dd>

<dt class="col-sm-3">Phone</dt>
<dd class="col-sm-9">@Model.Phone</dd>

<dt class="col-sm-3">Course</dt>
<dd class="col-sm-9">@Model.Course</dd>

<dt class="col-sm-3">Admission Date</dt>
<dd class="col-sm-9">@Model.AdmissionDate.ToString("dd MMMM yyyy")</dd>
</dl>
</div>
</div>
<form asp-action="Delete" method="post" class="mt-3">
    <input type="hidden" asp-for="StudentId" />
    <button type="submit" class="btn btn-danger">Yes, Delete</button>
    <a asp-action="Index" class="btn btn-secondary">Cancel</a>
</form>

```

OUTPUT:

```

Info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7011
Info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5285
Info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
Info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
Info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\luerthi\OneDrive\Desktop\upGrad\DAY-31\StudentManagementSystem\StudentManagementSystem
Info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (58ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [s].[StudentId], [s].[AdmissionDate], [s].[Course], [s].[Email], [s].[Name], [s].[Phone]
      FROM [Students] AS [s]
Info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (5ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [s].[StudentId], [s].[AdmissionDate], [s].[Course], [s].[Email], [s].[Name], [s].[Phone]
      FROM [Students] AS [s]
Info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (3ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [s].[StudentId], [s].[AdmissionDate], [s].[Course], [s].[Email], [s].[Name], [s].[Phone]
      FROM [Students] AS [s]
Info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (46ms) [Parameters=[@_student_Email_0='?' (Size = 100)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [s].[StudentId], [s].[AdmissionDate], [s].[Course], [s].[Email], [s].[Name], [s].[Phone]
      FROM [Students] AS [s]
      WHERE [s].[Email] = @_student_Email_0
Info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (4ms) [Parameters=[@p0='?' (DbType = DateTime2), @p1='?' (Size = 50), @p2='?' (Size = 100), @p3='?' (Size = 100), @p4='?' (Size = 100)], CommandType='Text', CommandTimeout='30']
      SET IMPLICIT_TRANSACTIONS OFF;
      SET NOCOUNT ON;
      INSERT INTO [Students] ([AdmissionDate], [Course], [Email], [Name], [Phone])
      OUTPUT INSERTED.[StudentId]
      VALUES (@p0, @p1, @p2, @p3, @p4);
Info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (2ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [s].[StudentId], [s].[AdmissionDate], [s].[Course], [s].[Email], [s].[Name], [s].[Phone]
      FROM [Students] AS [s]

```

Student Management System

Add New Student

Search

Clear

StudentId	Full Name	Email Address	Phone Number	Course Name	Admission Date	Actions
1	John Doe	john@example.com	9876543210	Computer Science	15-Jan-2024	Details Edit Delete
2	Jane Smith	jane@example.com	9876543211	Mathematics	20-Jan-2024	Details Edit Delete
3	Keerthivasan R	keerthivasankeerthi43@gmail.com	5555599999	AI	31-Mar-2026	Details Edit Delete

Add New Student

Full Name

Keerthivasan

Email Address

keerthivasankeerthi43@gmail.com

Phone Number

9025877719

Course Name

Cyber Security

Admission Date

31-03-2026

Save Student

Cancel